

Linux Embedded Systems

Lab Report

Student	Sylvan Arnold
Program	Computer Science
Academic year	2026
Class	MA_CSEL1
Repository	https://github.com/SylvanArnold/csel-workspace

1. Introduction

This report summarizes my work on the lab exercises for the “Linux embedded systems” course. Here is the link to the GitHub repository containing all my code:

GitHub Repository

Under the `src` directory, you can find the code for each exercise organized in separate folders. The `report` directory contains the source files for this report, including this main file (`main.typ`). The `teacher_solutions` directory contains the solutions provided by the teacher for each exercise.

2. Environment setup

2.1. Steps

I followed the steps as described in the lab instructions. Everything worked as expected.

Note: I added a rootfs overlay directory to the buildroot configuration to keep my custom files (like `/etc/fstab`). These files are included in the generated rootfs, so I don't have to modify the rootfs every time I rebuild the image.

2.2. Answers to Questions

2.2.1. How do we proceed to generate U-Boot?

In our case, U-Boot is already included in the buildroot configuration (via `make menuconfig`). The `make` command will generate the SD card image with U-Boot included.

2.2.2. How can we add additional packages to the buildroot configuration?

By running `make menuconfig` in the buildroot directory, we can access the configuration menu. Then, you can go inside the `Target packages` section and add additional packages to the build. If we run `make`, the new package will be included in the build.

2.2.3. How do we proceed to modify the kernel configuration?

By running `make linux-menuconfig` in the buildroot directory, we can access the kernel configuration menu and modify the kernel configuration.

2.2.4. How can we generate our own rootfs?

You can create a rootfs overlay directory, add it to the buildroot configuration (using `menuconfig`), and then when running `make`, the overlay directory will be included in the generated rootfs.

2.2.5. How can we use eMMC instead of an SD card?

Change the U-Boot boot command to load the kernel/rootfs from eMMC instead of the SD card (for example, change `ext4load mmc 1:1` to `ext4load mmc 0:1`). Use a USB flashing tool to flash the generated image directly onto the eMMC.

2.2.6. What would be the optimal configuration for developing applications in user space only?

The optimal configuration is to cross-compile the application on a host machine and then sync the built application with the target device. The easiest way to sync files is to use a CIFS share on the host machine and mount it on the target device (as we did with the workspace directory). Then, use SSH to access the target device and run the application. There is no need to sync the entire rootfs or download the kernel from the network at boot.

3. Kernel Modules Programming

3.1. Exercise 1

3.1.1. Create a skeleton module outside the kernel tree

I created a `skeleton.c` and a `Makefile`, inspired by the examples provided in the course. I built the module using `make`, which generated a `skeleton.ko` file.

3.1.2. Test the `modinfo` command on the generated module

The `modinfo` tool was not installed, so I added it to the buildroot configuration and rebuilt the image. Then, I replaced the `/rootfs` directory with the newly built one.

Here is the result:

```
filename:      /workspace/./src/02_modules/exercice01/mymodule.ko
license:      GPL
description:   Module skeleton
author:       Sylvan Arnold <sylvan.arnold@hes-so.ch>
depends:
name:         mymodule
vermagic:     5.15.148 SMP preempt mod_unload aarch64
parm:         text:charp
parm:         elements:int
```

3.1.3. Install the module and check `dmesg`

```
insmod /workspace/./src/02_modules/exercice01/mymodule.ko
dmesg
```

```
[ 761.493297] Linux module skeleton loaded
```

The module works as expected.

3.1.4. Compare `lsmod` and `cat /proc/modules` commands

The two commands give similar results. The `lsmod` command is a user-friendly way to display the loaded modules, while `cat /proc/modules` gives more detailed information.

3.2. Exercise 2

3.2.1. Adapt rootfs to include modprobe

I already installed modprobe in the rootfs in the previous exercise. The module also has to be installed in the rootfs. To do that, we can add an `install` target in the module's Makefile:

```
install:
    $(MAKE) -C $(KDIR) M=$(PWD) INSTALL_MOD_PATH=$(MODPATH) modules_install
```

Then, when we run `make install` in the module directory, the module will be installed in the rootfs.

3.2.2. Adapt the previous module to accept and log parameters

Parameters can be declared as static variables in the module's code:

```
#include <linux/moduleparam.h> /* needed for module parameters */

static char* text = "dummy text";
module_param(text, charp, 0664);
static int elements = 1;
module_param(elements, int, 0);
```

Then, we can log them in the console using `pr_info`:

```
pr_info("  text: %s\n", text);
pr_info("  elements: %d\n", elements);
```

The parameters can be passed to the module at load time using modprobe:

```
modprobe mymodule text="hello world" elements=42
```

The parameters are then displayed in `dmesg`:

```
[ 4564.063892] Linux module 01 skeleton loaded
[ 4564.068128]   text: hello world
[ 4564.068128]   elements: 42
```

3.3. Exercise 3

```
cat /proc/sys/kernel/printk
7      4      1      7
```

This command displays the current settings of the `printk` function. The first value is the current log level, the second value is the default log level attributed to prints that do not specify it, the third value is the minimum log level allowed, and the fourth value is the default log level set at boot.

3.4. Exercise 4

3.4.1. Memory allocation in the module

In the module's code, we can use the `kmalloc` function to allocate memory in the kernel space. The allocated memory can be freed using the `kfree` function. `linux/list.h` provides a linked list implementation.

3.4.2. My solution

I created code that creates a linked list of `x` elements at module installation and deletes it at module removal. Each element is a structure that contains an integer, a text, and a `list_head` structure. The integer is initialized with the element's index, and the text is initialized with a default value.

Here is the `dmesg` result of my module's installation/removal:

```
[ 97.216060] Sylvan Module Loaded
[ 97.219318] Creating list of 10 elements
[ 130.972082] Deleting element 0
[ 130.972108] Deleting element 1
[ 130.975174] Deleting element 2
[ 130.978268] Deleting element 3
[ 130.981373] Deleting element 4
[ 130.984451] Deleting element 5
[ 130.987516] Deleting element 6
[ 130.990569] Deleting element 7
[ 130.993633] Deleting element 8
[ 130.996696] Deleting element 9
[ 130.999759] Deleting element 10
[ 131.031133] Deleted 10 elements
[ 131.037425] Sylvan Module Unloaded
```

3.5. Exercise 5

I started by requesting the memory regions I needed:

```
ressources[0] = request_mem_region (0x01c14200, 0x10, "Chip ID");
ressources[1] = request_mem_region (0x01c25080, 0x4, "CPU Temperature");
ressources[2] = request_mem_region (0x01c30050, 0x8, "MAC Address");
```

But all these requests were failing. These memory regions seem to be already reserved. That's what I found by running `cat /proc/iomem`:

```
01c14000-01c143ff : 1c14000.eeprom eeprom@1c14000
01c25000-01c253ff : 1c25000.thermal-sensor thermal-sensor@1c25000
01c30000-01c3ffff : 1c30000.ethernet ethernet@1c30000
```

So, I changed the code to read these registers without claiming them. I used `ioremap` to map the physical memory addresses to virtual addresses, and then I read the values using `ioread32`.

Here is the dmesg result:

```
[ 551.289950] Chip ID: 82800001'94004704'5036c304'0425090e
[ 551.295290] CPU Temperature: 38753 m°C
[ 551.299132] MAC Address: 02:01:83:c6:1a:9b
```

I controlled the chip temperature with `cat /sys/class/thermal/thermal_zone0/temp` and also checked my MAC address with `ifconfig`. It matched the values read in the register.

3.6. Exercise 6

I created a thread in the module that prints a message every 5 seconds. I used `kthread_run` to create and run the thread and `kthread_stop` to stop it at module removal. The thread function is a simple loop that prints a message and sleeps for 5 seconds with `ssleep`.

Here is the dmesg result of the installation and removal of the module:

```
[ 3995.237292] Exercice 6 module loaded, reading registers...
[ 3995.243064] Thread is running
[ 4000.479559] tick
[ 4005.599597] tick
[ 4010.719563] tick
[ 4015.839569] tick
[ 4020.959569] tick
[ 4026.079559] tick
[ 4031.199560] tick
[ 4036.319555] tick
[ 4036.321465] Exercice 6 Module Unloaded, thread stopped
```

3.7. Exercise 7

I used a wait queue as requested in the instructions. At first, I had some issues understanding how to use it, as I was confusing wait queues with message queues. I created two threads: one that sleeps for 5 seconds and then wakes up the other thread using the `wake_up_interruptible` function. The second thread is sleeping on the wait queue using the `wait_event_interruptible` function and is woken up by the first thread or if the module is being unloaded.

The dmesg result:

```
[ 1750.174073] Exercice 7 module loaded, reading registers...
[ 1750.179928] Thread 1 is running
[ 1750.179967] Thread 2 is running
[ 1755.359354] Thread 1: notifying thread 2
[ 1755.363350] Thread 2: received notification from thread 1
[ 1760.479348] Thread 1: notifying thread 2
[ 1760.483296] Thread 2: received notification from thread 1
[ 1765.599346] Thread 1: notifying thread 2
[ 1765.603288] Thread 2: received notification from thread 1
...
[ 1780.872832] Exercice 7 Module Unloaded, threads stopped
```

3.8. Exercise 8

I created a single interrupt handler for the three buttons. To distinguish the buttons, I created an **enum** that represents the three buttons and passed the corresponding value for each button.

The dmesg result:

```
[10209.745763] Failed to request IRQ for GPIO 0  
[10209.750235] Exercice 8 module loaded  
[10248.209782] Interrupt from K2  
[10254.388959] Interrupt from K3  
[10264.665848] Interrupt from K2
```


4. Device Drivers

4.1. Exercise 1

I didn't know how to create the Makefile for a memory-mapped device driver, so I analyzed and used the one provided in the solution. I wrote some comments in the Makefile to explain what each part is doing.

The same goes for the `main.c` file. I didn't know where to start, as it was my first time writing a memory-mapped device driver. I analyzed and commented on the solution to understand how it works.

The result of the running app:

```
psz=1000, addr=1c14200, offset=1c14000, ofs=200
NanoPi NEO Plus2 chipid=82800001'94004704'5036c304'0425090e
```

The example made me understand the importance of aligning memory addresses to the page size when using `mmap` to map physical memory to user space.

4.2. Exercise 2

I created a character device driver based on the skeleton example in the course. I implemented the `open`, `release`, `read`, and `write` functions.

The module has an internal `char*` pointer initialized with a null value. When the user writes to the device, the pointer is allocated with `kmalloc`, and the value is copied from user space to kernel space using `copy_from_user`. The old value is also freed using `kfree` before allocating the new one if it is not the first write. When the user reads from the device, the value is copied from kernel space to user space using `copy_to_user`.

I installed the module with `modprobe` and inspected `dmesg` to check the major and minor numbers that were attributed:

```
[ 533.612204] skeleton: allocated char device region with major 511 and
minor 0
```

Then, I created a device file with `mknod`:

```
mknod /dev/skeleton c 511 0
ls -l /dev/skeleton
```

The result:

```
crw-r--r-- 1 root root 511, 0 Jan  1 00:16 /dev/skeleton
```

Then, I tested the device with the `echo` and `cat` commands:

```
echo "Hello world" > /dev/skeleton  
cat /dev/skeleton
```

It worked as expected, even after multiple writes.

4.3. Exercise 3

To set the device numbers, I added a `device_n` module parameter with a default value of 1 that the user can change at load time. Then, in the `init` function, I used a loop that creates `n` devices.

I had to adapt the previous code to handle multiple devices. To make things much simpler, I defined a fixed buffer size that is created for each device at module initialization and freed once at module removal.

I also modified the code to directly create the device files in `/dev` using the `device_create` function instead of creating them manually with `mknod`.

4.4. Exercise 4

I created a Rust program that interacts with the devices created in the previous exercise. Here are the commands I used to compile it for the correct architecture:

```
rustup target add aarch64-unknown-linux-gnu
export PATH=/buildroot/output/host/usr/bin:$PATH
cargo build --release --target aarch64-unknown-linux-gnu
```

Then, I ran the program:

```
/workspace/src/03_drivers/exercise04/target/aarch64-unknown-linux-gnu/
release/exercise04
```

The program's output:

```
Opened all devices
Response from device 0: Hello, device 0
Response from device 1: Hello, device 1
Response from device 2: Hello, device 2
```

4.5. Exercise 5

I created a simple class device with `sysfs` attributes. I struggled to understand where `&dev_attr_data` and `&dev_attr_cfg` were coming from. I found out that they are created by the `DEVICE_ATTR` macro that we use.

4.6. Exercise 5.1

I added the character device file operations developed in Exercise 2 to the module. Then, I installed the module with `modprobe` and tested it.

I tested the character device:

```
ls -l /dev/ex5_device
echo "Hello world" > /dev/ex5_device
cat /dev/ex5_device
```

Result:

```
crw-rw---- 1 root root 511,  0 Jan  1 01:10 ex5_sysfs_device
Hello world
```

And the sysfs attributes:

```
echo 77 | tee /sys/class/ex5_device_class/ex5_sysfs_device/sysfs_data
cat /sys/class/ex5_device_class/ex5_sysfs_device/sysfs_data
```

Result:

```
77
```

```
echo "7 12345 demo cfg" | tee /sys/class/ex5_device_class/ex5_sysfs_device/cfg
cat /sys/class/ex5_device_class/ex5_sysfs_device/cfg
```

Result:

```
7 12345 demo cfg
```

The driver works as expected. The character device and the sysfs attributes are working correctly.

4.7. Exercise 7

I used my Section 3.8 solution as a basis for this exercise.

The driver caller has multiple options: he can call the `read` function in blocking mode and wait for an interrupt to be triggered, or he can call it in non-blocking mode and check if an interrupt has been triggered without waiting. He can use the `poll` function to wait for an interrupt to be triggered and be notified when it happens. I also exposed a sysfs attribute to reset the interrupt counter.

I developed a small Rust app that tests the driver. It resets the interrupt counter, then calls the `poll` function to wait for an interrupt to be triggered. When an interrupt is triggered, it reads the number of interrupts and adds it to the total interrupts counter. It also prints the number of interrupts received from the driver and the total interrupts.